

Week 2

Game Engine Development Unity AI Navigation

Hooman Salamat

Objective & Concept

Objective

- Building Intelligent Pathfinding for Games & Interactive Experiences

Concepts

- Explore Unity's modern AI Navigation package, learn how to build walkable surface geometry (NavMesh), and script intelligent agent behaviors.

What is NavMesh?

Definition: A simplified 2D/3D polygon mesh generated over scene geometry to define walkable regions.

Purpose: Allows AI characters to calculate the shortest path and navigate complex environments without colliding with obstacles.

Core Package: Unity AI Navigation Package (com.unity.ai.navigation).

The AI Navigation package contains high-level components that allow you to use navmeshes to incorporate navigation and pathfinding in your game.

With this package installed you can build and use navmeshes at runtime and at edit time, create dynamic obstacles, and use links to allow specific actions (like jumping) as your characters navigate between navmeshes.



Inner Workings of the Navigation System

When you want to intelligently move characters in your game (or agents as they are called in AI circles), you have to solve two problems:

how to reason about the level to find the destination,

then how to move there. These two problems are tightly coupled, but quite different in nature.

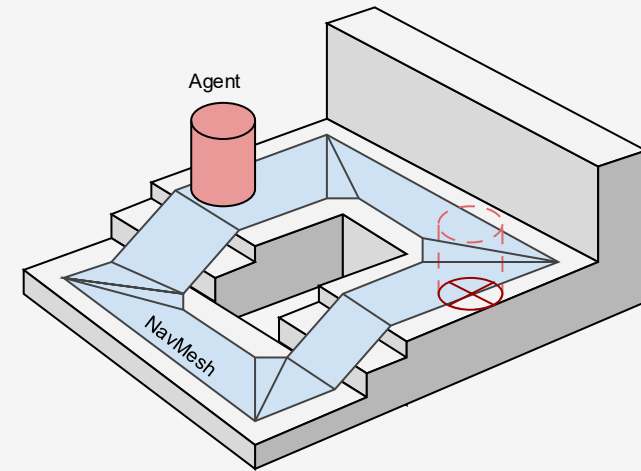
The problem of reasoning about the level is more global and static, in that it takes into account the whole [Scene](#).

Moving to the destination is more local and dynamic, it only considers the direction to move and how to prevent [collisions](#) with other moving agents.



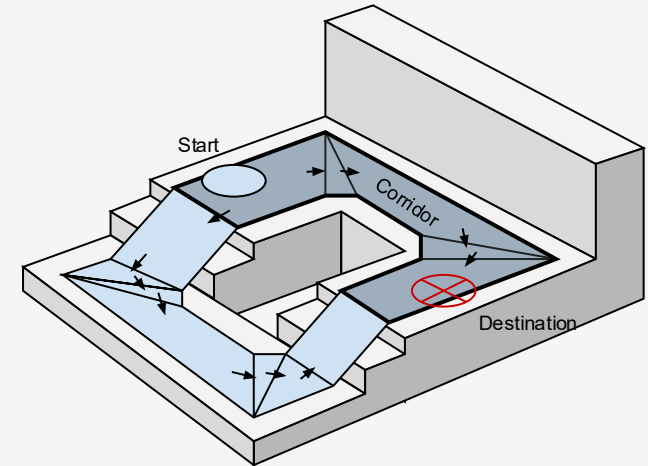
Walkable Areas

- The navigation system needs its own data to represent the walkable areas in a game scene.
- The walkable areas define the places in the scene where the agent can stand and move.
- In Unity the agents are described as cylinders.
- The walkable area is built automatically from the geometry in the scene by testing the locations where the agent can stand.
- Then the locations are connected to a surface laying on top of the scene geometry.
- This surface is called the navigation [mesh](#) (NavMesh for short).
- The [NavMesh](#) stores this surface as convex polygons.
- Convex polygons are a useful representation, since we know that there are no obstructions between any two points inside a polygon.
- In addition to the polygon boundaries, we store information about which polygons are neighbors to each other. This allows us to reason about the whole walkable area.



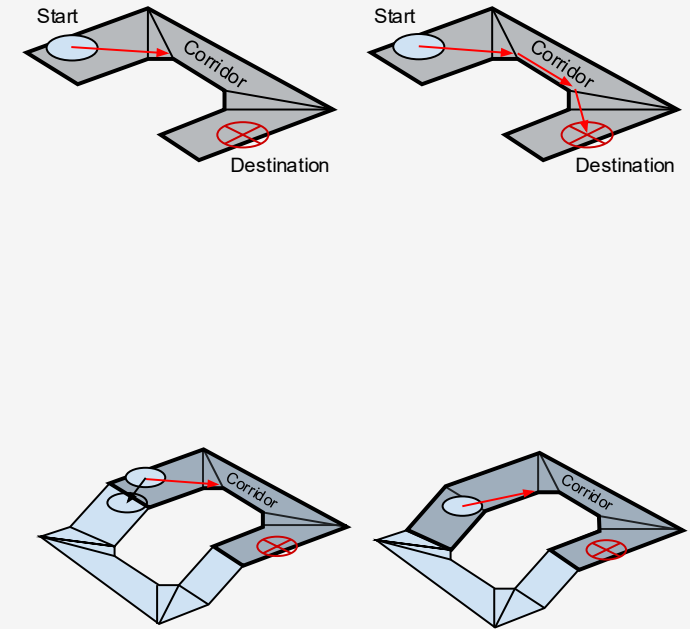
Finding Paths

- To find a path between two locations in the scene, we first need to map the start and destination locations to their nearest polygons.
- Then we start searching from the start location, visiting all the neighbors until we reach the destination polygon.
- Tracing the visited polygons allows us to find the sequence of polygons which will lead from the start to the destination.
- A common algorithm to find the path is A* (pronounced “A star”), which is what Unity uses.



Following the Path

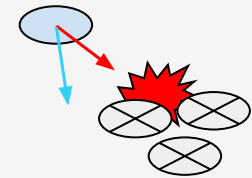
- The sequence of polygons which describe the path from the start to the destination polygon is called a corridor. The agent will reach the destination by always steering towards the next visible corner of the corridor. If you have a simple game where only one agent moves in the scene, it is fine to find all the corners of the corridor in one swoop and animate the character to move along the line segments connecting the corners.
- When dealing with multiple agents moving at the same time, they will need to deviate from the original path when avoiding each other. Trying to correct such deviations using a path consisting of line segments soon becomes very difficult and error prone.
- Since the agent movement in each frame is quite small, we can use the connectivity of the polygons to fix up the corridor in case we need to take a little detour. Then we quickly find the next visible corner to steer towards.



Avoiding Obstacles

The steering logic takes the position of the next corner and based on that figures out a desired direction and speed (or velocity) needed to reach the destination. Using the desired velocity to move the agent can lead to collision with other agents.

Obstacle avoidance chooses a new velocity which balances between moving in the desired direction and preventing future collisions with other agents and edges of the navigation mesh. Unity is using reciprocal velocity obstacles (RVO) to predict and prevent collisions.



Moving the Agent

Finally after steering and obstacle avoidance the final velocity is calculated. In Unity the agents are simulated using a simple dynamic model, which also takes into account acceleration to allow more natural and smooth movement.

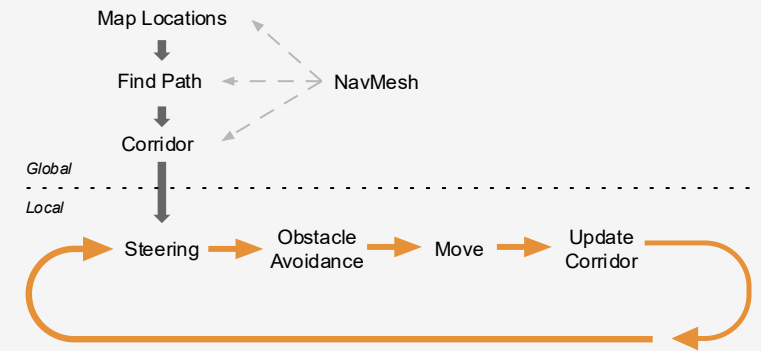
At this stage you can feed the velocity from the simulated agent to the animation system to move the character using [root motion](#), or let the navigation system take care of that.

Once the agent has been moved using either method, the simulated agent location is moved and constrained to NavMesh. This last small step is important for robust navigation.



Global and Local

- One of the most important things to understand about navigation is the difference between global and local navigation.
- Global navigation is used to find the corridor across the world. Finding a path across the world is a costly operation requiring quite a lot of processing power and memory.
- The linear list of polygons describing the path is a flexible data structure for steering, and it can be locally adjusted as the agent's position moves. Local navigation tries to figure out how to efficiently move towards the next corner without colliding with other agents or moving objects.



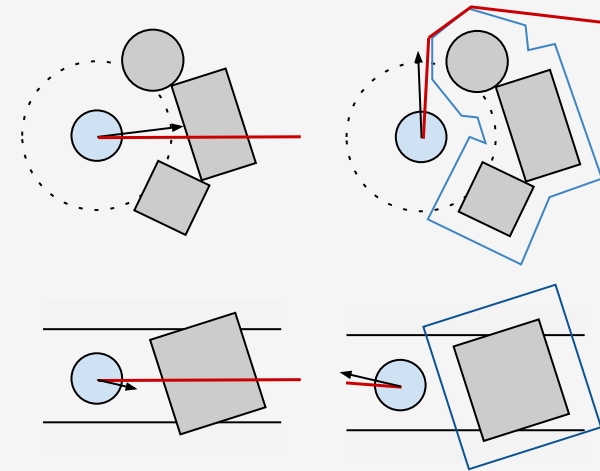
Two Cases for Obstacles

Many applications of navigation require other types of obstacles rather than just other agents. These could be the usual crates and barrels in a shooter game, or vehicles. The obstacles can be handled using local obstacle avoidance or global pathfinding.

When an obstacle is moving, it is best handled using local obstacles avoidance. This way the agent can predictively avoid the obstacle. When the obstacle becomes stationary, and can be considered to block the path of all agents, the obstacles should affect the global navigation, that is, the navigation mesh.

Changing the NavMesh is called carving. The process detects which parts of the obstacle touches the NavMesh and carves holes into the NavMesh. This is a computationally expensive operation, which is yet another compelling reason, why moving obstacles should be handled using collision avoidance.

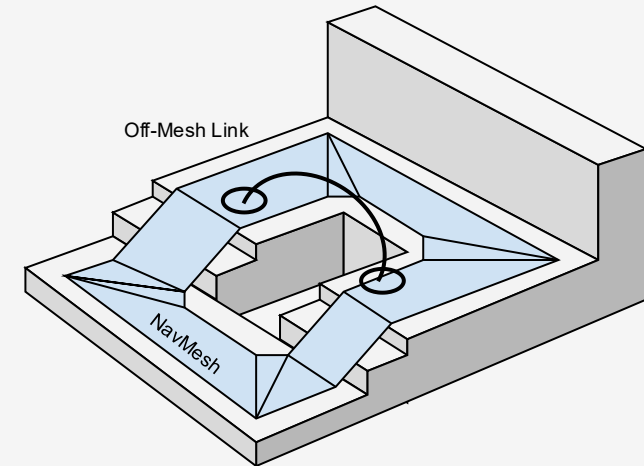
You can use local collision avoidance to steer around sparsely scattered obstacles too. Since the algorithm is local, it only considers the next immediate collisions, and cannot steer around traps or handle cases where an obstacle blocks a path. Use carving to solve these cases.



About shortcuts between positions on NavMeshes

The connections between the NavMesh polygons are described using links inside the pathfinding system. Sometimes it is necessary to let the agent navigate across places which are not walkable, for example, jumping over a fence, or traversing through a closed door. These cases need to know the location of the action.

You can annotate these actions with the **NavMesh Link** component, which tells the pathfinder that a route exists through the specified link. Your code can later access this link and perform the special action as the agent follows the path.



About Voxels

The NavMesh bake process uses voxelization to build the NavMesh from arbitrary level geometry. The algorithm first [rasterizes](#) the scene into voxels, then it extracts the walkable surfaces, then finally, turns the walkable surfaces into a navigation mesh. Voxels are the cells in a regular 3D grid where the scene geometry overlaps with that grid. The grid cells have a width and a length of [Voxel Size](#) and a height that is half of that width.

To increase the accuracy of the resulting NavMesh shape, reduce the **Voxel Size** so that the 3D grid can rasterize finer details from the scene geometry. The time it takes to process the scene geometry is proportional to the number of voxels that the geometry occupies. When the NavMesh Surface uses a small voxel size, it generally creates the NavMesh more slowly than when it uses a larger voxel size. If you want to create the NavMesh faster, and you can no longer reduce the number of [scene objects](#) to process, you can increase the voxel size. With that modification, the resulting NavMesh matches the scene geometry with lower accuracy, both at the edges around obstacles and in the elevation relative to the ground.

To extend the NavMesh Surface through narrow passages, such as doors, and to maintain a quick baking time, choose the voxel size such that 3 voxels fit one Agent radius (6 per diameter). The **NavMesh Surface** uses this size by default. For big open areas, using 1 or 2 voxels per radius speeds up baking. Tight indoor spots are better suited to smaller voxels, for example 4 to 6 voxels per radius. More than 8 voxels per radius does not usually provide much additional benefit.



About NavMesh agents

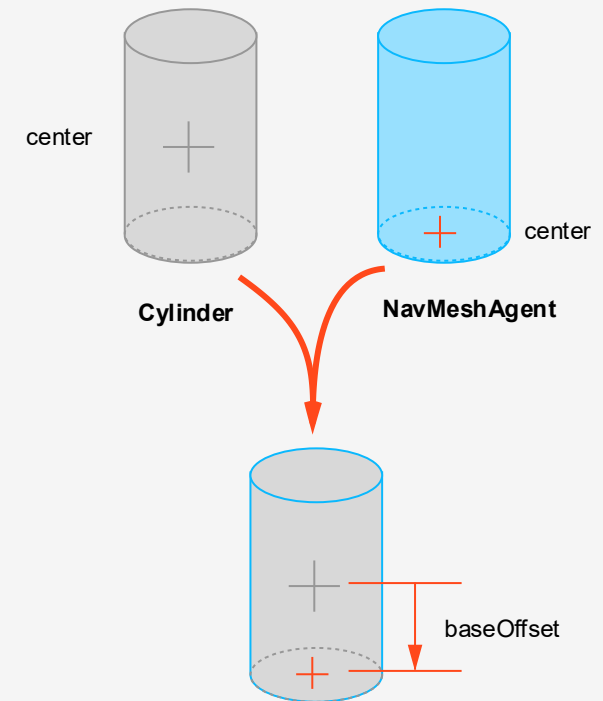
The NavMesh agent is a [GameObject](#) that is represented by an upright cylinder whose size is specified by the Radius and Height properties. The cylinder moves with the GameObject, but remains upright even if the GameObject rotates. The shape of the cylinder is used to detect and respond to collisions with other agents and obstacles. When the anchor point of the GameObject is not at the base of the cylinder, use the Base Offset property to specify the height difference.

The height and radius of the cylinder are specified in the [Navigation window](#) and the [NavMesh Agent component](#) properties of the individual agents.

Navigation window settings describe how all the NavMesh Agents collide and avoid static world geometry. To keep memory on budget and CPU load at a reasonable level, you can only specify one size in the bake settings.

NavMesh Agent component properties values describe how the agent collides with moving obstacles and other agents.

Typically you set the size of the agent with the same values in both places. However, you might, give a heavy soldier a larger radius, so that other agents leave more space around your soldier. Otherwise, your soldier avoids the environment in the same manner as the other agents.



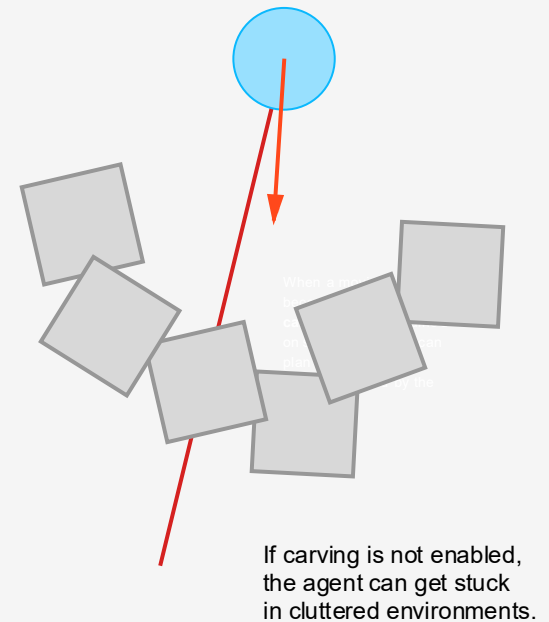
About NavMesh Obstacles

Obstructing

When **Carve** is not enabled, the default behavior of the NavMesh Obstacle is similar to that of a [Collider](#). NavMesh Agents try to avoid [collisions](#) with the NavMesh Obstacle, and when close, they collide with the NavMesh Obstacle. Obstacle avoidance behavior is very basic, and has a short radius. As such, the NavMesh Agent might not be able to find its way around in an environment cluttered with NavMesh Obstacles. This mode is best used in cases where the obstacle is constantly moving (for example, a vehicle or player character).

Carving

When **Carve** is enabled, the obstacle carves a hole in the NavMesh when stationary. When moving, the obstacle is an obstruction. When a hole is carved into the NavMesh, the pathfinder is able to navigate the NavMesh Agent around locations cluttered with obstacles, or find another route if the current path gets blocked by an obstacle. It's good practice to turn on carving for NavMesh Obstacles that generally block navigation but can be moved by the player or other game events like explosions (for example, crates or barrels).



Logic for moving NavMesh Obstacles

Unity treats the NavMesh Obstacle as moving when it has moved more than the distance set by the **Carve > Move Threshold**. When the NavMesh Obstacle moves, the carved hole also moves. However, to reduce CPU overhead, the hole is only recalculated when necessary. The result of this calculation is available in the next frame update. The recalculation logic has two options:

Only carve when the NavMesh Obstacle is stationary

Carve when the NavMesh Obstacle has moved

Only carve when the NavMesh Obstacle is stationary

This is the default behavior. To enable it, tick the NavMesh Obstacle component's **Carve Only Stationary** checkbox. In this mode, when the NavMesh Obstacle moves, the carved hole is removed. When the NavMesh Obstacle has stopped moving and has been stationary for more than the time set by **Carving Time To Stationary**, it is treated as stationary and the carved hole is updated again. While the NavMesh Obstacle is moving, the NavMesh Agents avoid it using collision avoidance, but don't plan paths around it.

Carve Only Stationary is generally the best choice in terms of performance, and is a good match when the [GameObject](#) associated with the NavMesh Obstacle is controlled by physics.

Carve when the NavMesh Obstacle has moved

To enable this mode, clear the NavMesh Obstacle component's **Carve Only Stationary** checkbox. When this is cleared, the carved hole is updated when the obstacle has moved more than the distance set by **Carving Move Threshold**. This mode is useful for large, slowly moving obstacles (for example, a tank that is being avoided by infantry).

Navigation Areas

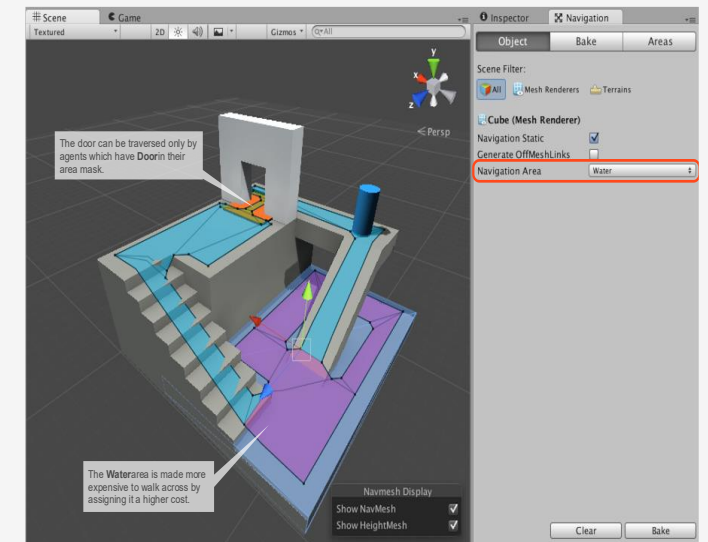
The **Navigation Area** defines how difficult it is to walk across a specific area. This is important for finding the path with the lowest total cost. In addition, each [NavMesh Agent](#) has an **Area Mask** which you can use to specify which areas the agent can move on.

In the example above the area types are used for two common use cases:

The **Water** area is made more costly to walk through by assigning it a higher cost, to deal with a scenario where walking in shallow water is slower.

The **Door** area is made accessible to specific characters, to create a scenario where humans can walk through doors, but zombies cannot.

You can assign the area type to every object that is included in the [NavMesh](#) baking. In addition, each **NavMesh Link** has a property to specify the area type.



Pathfinding Cost

The cost allows you to control which areas the pathfinder favors when finding a path. For example, if you set the cost of an area to 3.0, traveling across that area is considered to be three times longer than alternative routes.

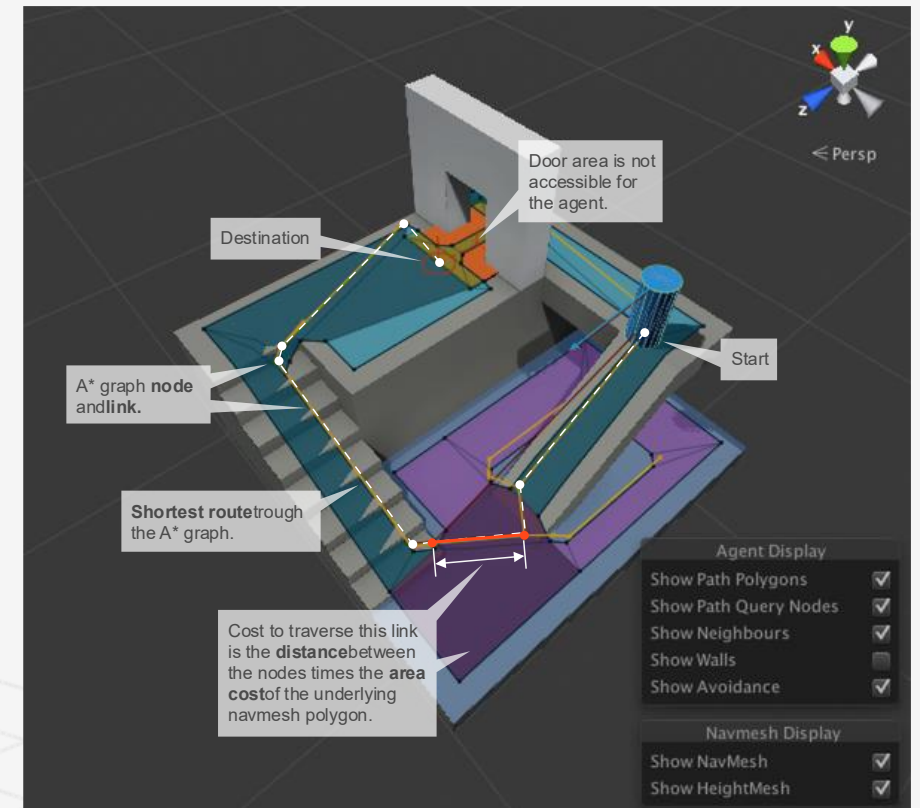
Unity uses A* to calculate the shortest path on the NavMesh. A* works on a graph of connected nodes. The algorithm starts from the nearest node to the path start and visits the connect nodes until the destination is reached.

Since the Unity navigation representation is a [mesh](#) of polygons, the first thing the pathfinder needs to do is to place a point on each polygon, which is the location of the node. The shortest path is then calculated between these nodes.

The yellow dots and lines in the above picture shows how the nodes and links are placed on the NavMesh, and in which order they are traversed during the A*.

The cost to move between two nodes depends on the distance to travel and the cost associated with the area type of the polygon under the link, that is, $distance * cost$. In practice this means, that if the cost of an area is 2.0, the distance across such polygon will appear to be twice as long. The A* algorithm requires that all costs must be larger than 1.0.

The effect of the costs on the resulting path can be hard to tune, especially for longer paths. The best way to approach costs is to treat them as hints. For example, if you want the agents to not use **NavMesh Links** too often, you could increase their cost. But it can be challenging to tune a behavior where the agents prefer to walk on sidewalks.



Area Types

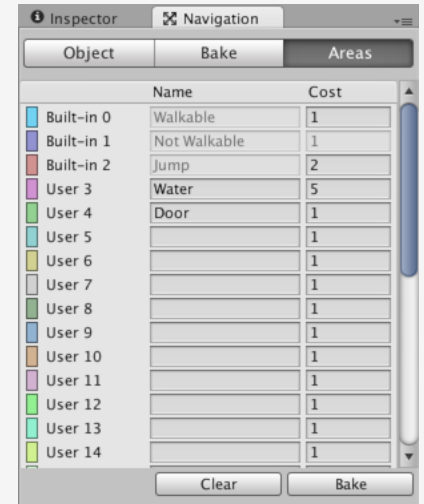
The area types are specified in the *Navigation Window's* *Areas* tab. There are 29 custom types, and 3 built-in types: *Walkable*, *Not Walkable*, and *Jump*.

Walkable is a generic area type which specifies that the area can be walked on.

Not Walkable is a generic area type which prevents navigation. It is useful for cases where you want to mark certain object to be an obstacle, but without getting NavMesh on top of it.

Jump is an area type that is assigned to all auto-generated **NavMesh Links**.

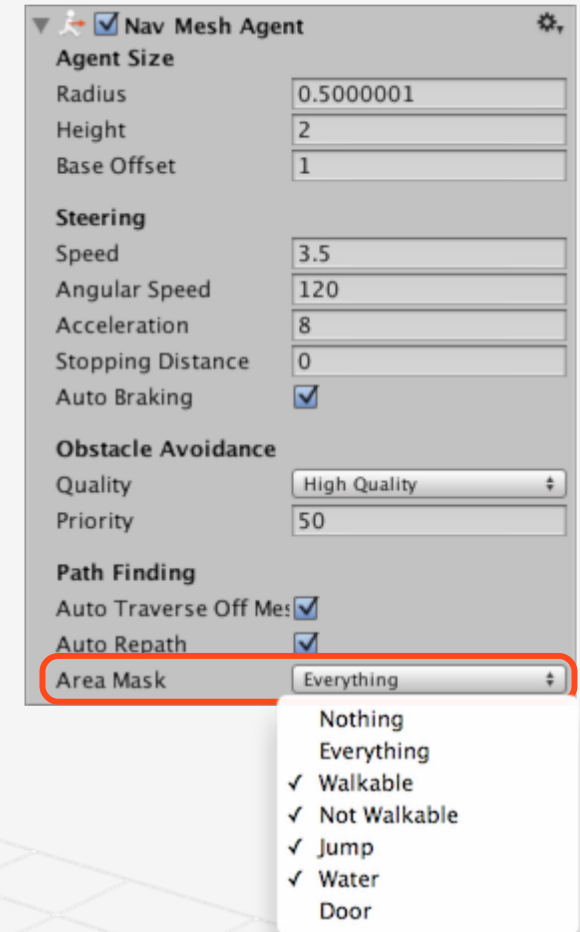
If several objects of different area types are overlapping, the resulting NavMesh area type will generally be the one with the highest index. There is one exception however: *Not Walkable* always takes precedence. Which can be helpful if you need to block out an area.



Area Mask

Each agent has an *Area Mask* which describes which areas it can use when navigating. The area mask can be set in the agent properties, or the bitmask can be manipulated using a script at runtime.

The area mask is useful when you want only certain types characters to be able to walk through an area. For example, in a zombie evasion game, you could mark the area under each door with a *Door* area type, and uncheck the Door area from the zombie character's Area Mask.



NavMesh Surface & Components

NavMeshSurface: Defines the walkable area for a specific Agent type and triggers the "Bake" process.

NavMeshAgent: Attached to moving characters to handle pathfinding and movement.

NavMeshObstacle: Attached to dynamic objects (e.g., falling crates, moving doors).

OffMeshLink / NavMeshLink: Connects disconnected NavMesh regions (e.g., jumps, ladders, teleporters).



Baking Process

Install Package: Open Window > Package Manager and install **AI Navigation**.

Create Surface: Add a NavMesh Surface component to an empty GameObject or environment parent.

Configure Agent Settings: Set Agent Radius, Agent Height, Max Slope, and Step Height.

Bake: Click **Bake** to generate the blue walkable surface in the Scene View.



Controlling the NavMesh

Import namespace:

```
using UnityEngine.AI;
```

Essential Method: `agent.SetDestination(Vector3 targetPosition);`

Key Properties: `speed`, `stoppingDistance`, `remainingDistance`,
`isStopped`

```
using UnityEngine;
using UnityEngine.AI;

public class EnemyAI : MonoBehaviour {
    public Transform target;
    private NavMeshAgent agent;

    void Start() {
        agent = GetComponent<NavMeshAgent>();
    }

    void Update() {
        if (target != null) {
            agent.SetDestination(target.position);
        }
    }
}
```

Navigating Dynamic Environments

Dynamic Obstacles: Use NavMeshObstacle with "Carve" enabled to cut temporary holes in the NavMesh at runtime.

Off-Mesh Links: Set start and end points for jumping across chasms or climbing drop-offs.

Area Costs: Assign higher traversal costs to specific zones (e.g., mud, shallow water) to force AI to prefer paved paths.



Designing State Machines for AI

Patrolling: Moving between an array of waypoints.

Chasing: Triggered when the player enters line-of-sight or detection radius.

Searching / Fleeing: Repositioning away from danger or searching last known positions.



Performance Optimization & Troubleshooting

Agent Sticking / Clipping: Adjust Radius and Height settings in NavMeshAgent to match the model bounds.

Performance Optimization: Avoid calling `SetDestination()` every frame; throttle updates using timers or distance checks.

Multiple Agent Types: Bake custom NavMeshes for different entity sizes (e.g., small humanoid vs. giant vehicle).



Create a NavMesh

You need to create a [NavMesh](#) to define an area of your scene within which a character can navigate intelligently.

To create a NavMesh do the following:

Select the scene geometry where you want to add the NavMesh.

In the Inspector window, click **Add Component**.

Select **Navigation > NavMesh Surface**.

In the NavMesh Surface component, specify the necessary settings. For details on the available settings, refer to [NavMesh Surface component](#).

When you are finished, click **Bake**.

The NavMesh is generated and displayed in the scene as a blue overlay on the underlying scene geometry whenever the Navigation window is open and visible.

You can bake the NavMesh again to update it each time you make changes to either the scene geometry, the NavMesh [modifiers](#), the properties of the **NavMesh Surface** component, or [the settings](#) of [the selected agent type](#).

To permanently remove a NavMesh from your project, do one of the following:

Click the **Clear** button in the **NavMesh Surface** inspector.

Delete [the NavMesh asset file](#) in the **Project** window. If you choose to remove the **NavMesh Surface** component itself from the GameObject, the asset file is not deleted, even though the NavMesh is no longer present in the scene.

Create a NavMesh Agent

Once you have a [NavMesh](#) baked for your level it is time to create a character which can navigate the [Scene](#). We're going to build our prototype agent from a cylinder and set it in motion. This is done using a NavMesh Agent component and a simple script.

Create a **cylinder**: **GameObject > 3D Object > Cylinder**.

The default cylinder dimensions (height 2 and radius 0.5) are good for a humanoid shaped agent, so we will leave them as they are.

Add a **NavMesh Agent** component: **Component > Navigation > NavMesh Agent**.

Now you have simple NavMesh Agent set up ready to receive commands!

When you start to experiment with a NavMesh Agent, you most likely are going to adjust its dimensions for your character size and speed.

The **NavMesh Agent** component handles both the pathfinding and the movement control of a character. In your [scripts](#), navigation can be as simple as setting the desired destination point – the NavMesh Agent can handle everything from there on.

Next we need to build a simple script which allows you to send your character to the destination specified by another Game Object, and a Sphere which will be the destination to move to:

Create a new **C# script** (MoveTo.cs) and replace its contents with the above script.

Assign the MoveTo script to the character you've just created.

Create a **sphere**, this will be the destination the agent will move to.

Move the sphere away from the character to a location that is close to the NavMesh surface.

Select the character, locate the MoveTo script, and assign the Sphere to the **Goal** property.

Press Play; you should see the agent navigating to the location of the sphere.

```
// MoveTo.cs

using UnityEngine;
using UnityEngine.AI;

public class MoveTo :
MonoBehaviour {

    public Transform goal;

    void Start () {

        NavMeshAgent agent =
GetComponent<NavMeshAgent>
(
);

        agent.destination =
goal.position;

    }

}
```

Create a NavMesh Obstacle

NavMesh Obstacle components can be used to describe obstacles the agents should avoid while navigating. For example the agents should avoid physics controlled objects, such as crates and barrels while moving.

First create a **cube** to depict the crate: **Game Object > 3D Object > Cube**.

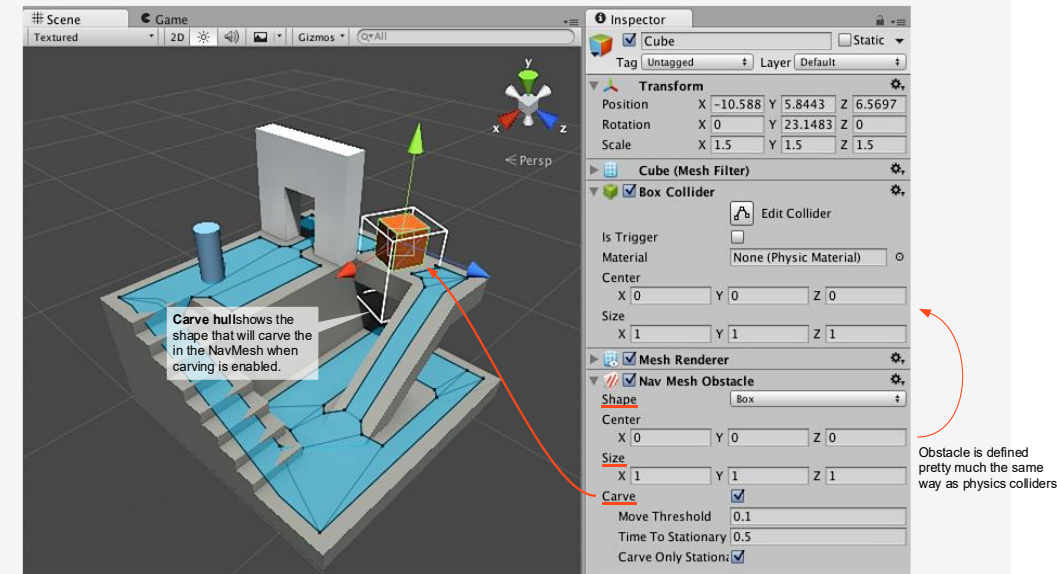
Move the cube to the platform at the top, the default size of the cube is good for a crate so leave it as it is.

Add a **NavMesh Obstacle** component to the cube. Choose **Add Component** from the inspector and choose **Navigation > NavMesh Obstacle**.

Set the shape of the obstacle to **box**, changing the shape will automatically fit the center and size to the render [mesh](#).

Add a **Rigid body** to the obstacle. Choose **Add Component** from the inspector and choose **Physics > Rigid Body**.

Finally turn on the **Carve** setting from the [NavMesh](#) Obstacle [inspector](#) so that the agent knows to find a path around the obstacle.



Create a NavMesh Link

Use NavMesh Links when you want to create paths that cross outside of the walkable [navigation mesh](#) surface. For example, you can set up NavMesh Links to allow agents to jump over a ditch or a fence, or to open a door before walking through it. To do this, you first need to add a NavMesh Link component to a GameObject in your [scene](#).

First create two cylinders: **Game Object > 3D Object > Cylinder**.

Scale the cylinders to (0.1, 0.05, 0.1) to make it easier to work with them.

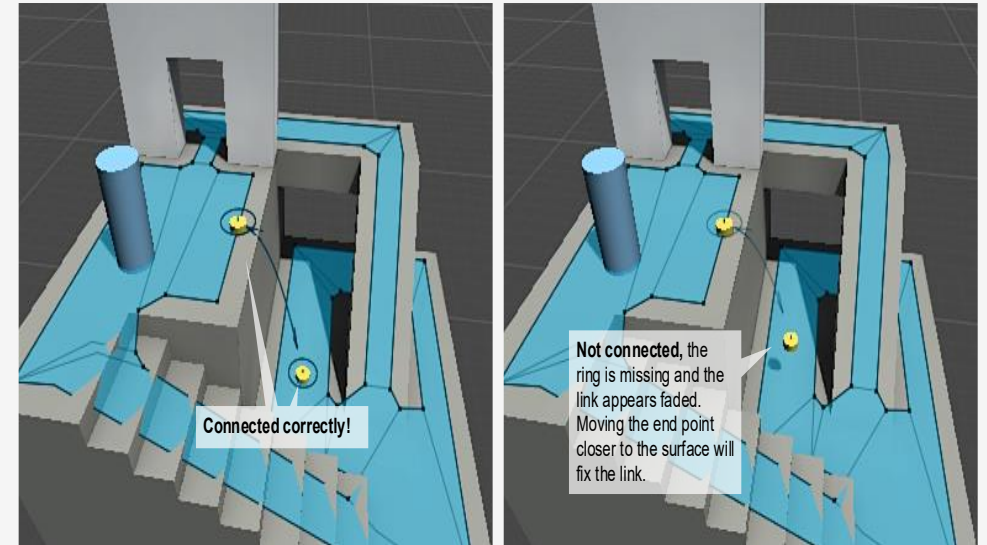
Move the first cylinder to the edge of the top platform, close to the [NavMesh](#) surface.

Place the second cylinder on the ground, close to the NavMesh, at the location where the link should land.

Select the first cylinder and add a **NavMesh Link** component to it. In the inspector, select **Add Component > Navigation > NavMesh Link**.

In the **Start Transform** field, assign the first cylinder.

In the **End Transform** field, assign the second cylinder.



Summary & Next Steps

Key Takeaways: NavMesh automates complex pathfinding, allowing you to focus on high-level AI state logic and gameplay loop mechanics.

Suggested Assignment: Build a simple maze project with a patrolling guard that switches to chasing the player when detected.

